



Distributed, GPU-Aware Programming with FleCSI

FleCSI tutorial

LA-UR-25-32044

2026

Module 1

Introduction and Terminology



The Growing Complexity of HPC Systems

- Latest top HPC systems exhibit millions of CPU cores, GPUs from different vendors, and specialized accelerators.
- Increasing system size and heterogeneity makes achieving peak performance extremely challenging.

Rank	System	Cores
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128

Challenges Across System Layers

- Mastering hardware, memory hierarchy, and distributed parallelism demands broad technical expertise.
- Domain scientists risk being diverted from scientific goals by low-level system complexities.
- Continuous collaboration between domain scientists and architectural experts is extremely expensive.

What is FleCSI?



- enables **single-source performance-portable parallel and distributed codes**.
- **supports multiple distributed backends:** Legion, HPX, and MPI.
- **supports on-node parallelism** for CPU and GPU via Kokkos.
- provides abstraction layers to **promote code reuse** and allow domain scientists to **focus only on their methods and their data usage**.

Related Work and Positioning

- **Legion**: data-centric, task-based runtime using logical regions and control replication for scalable parallel execution.
 - **FleCSI** leverages Legion as an optional backend while maintaining a unified single-source API across MPI and HPX.
- **StarPU**: heterogeneous task scheduler using explicit data handles and performance models.
 - **FleCSI** compiles task and data registration statically, minimizing runtime overhead and eliminating manual scheduling.
- **Uintah**: component-based multi-physics framework relying on task graphs and AMR.
 - **FleCSI** provides a general task/data abstraction applicable beyond specific physics domains and supports interchangeable backends.

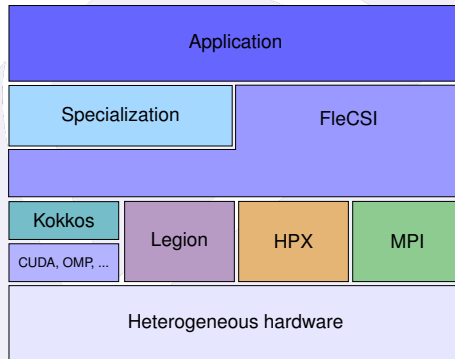
Tutorial Objectives

- Introduction
 - Introduction and Terminology
 - Flog
- Control Model & Runtime Model
 - Actions, Cycles
 - Runtime
- Data Model
 - Concepts (Index Spaces, Fields)
 - Fields (Declare fields, Layouts)
 - Topologies (Register fields)
- Execution Model
 - Tasks (Privileges, Usage)
 - Futures, Reductions
- On-Node Parallelism
 - Executors, Kernels
 - Portable Tasks
- *Extending FleCSI*
 - *Writing Specializations*
 - *Task Interfaces*

FleCSI Terminology

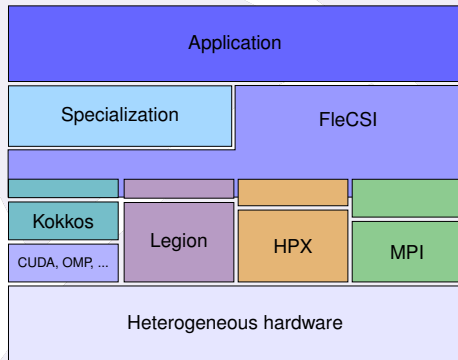
Structure of FleCSI Applications

- **Applications** care only about the physics and the data accesses.
- **Specialization** takes advantage of FleCSI to create domain-specific interfaces.
- **FleCSI** allocates distributed data and dispatches parallel work on it using lower-level interfaces.



Backends

- **The MPI backend provides**
 - Distributed-memory execution across processes.
 - Bulk synchronous parallelism and ghost-data exchange.
- **The HPX backend adds**
 - Asynchronous task execution with parallelism discovery.
 - Fine-grained scheduling integrated with on-node parallelism.
- **The Legion backend adds**
 - Region-based task model with data-locality awareness.
 - Support for cross-color data access.

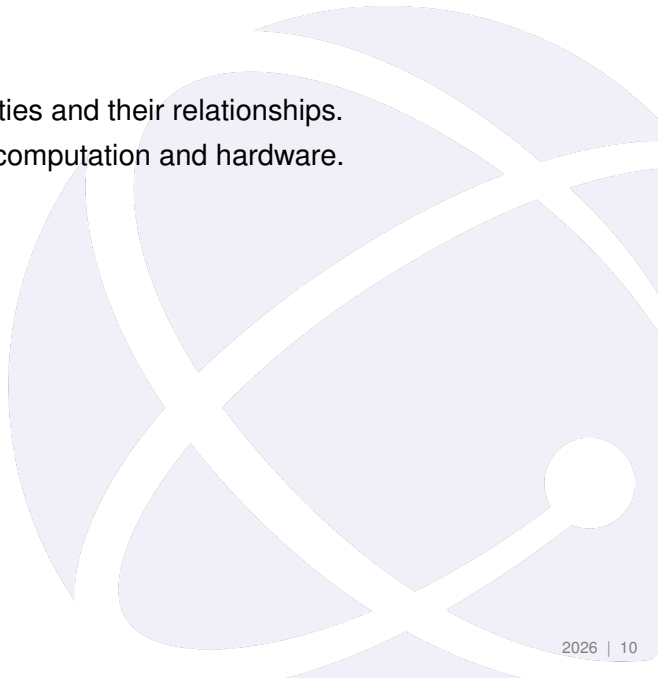


Topologies

- Abstract structure of simulation entities and their relationships.
- Problem definition decoupled from computation and hardware.

Topologies exist for

- matrices
- structured Cartesian meshes
- unstructured meshes
- interacting particles or mesh tiles
- simulation metadata and statistics



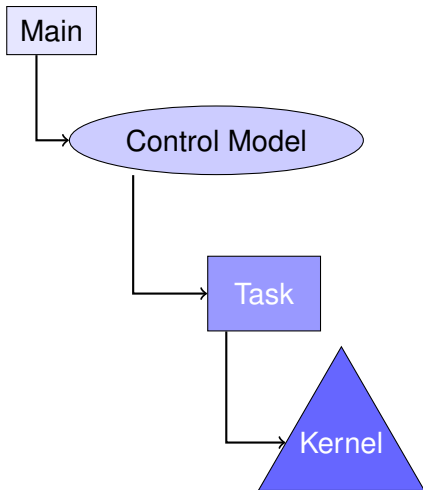
Specializations

- Defines interface that allows domain experts to implement numerical methods on top of the topology.
- The relevant operations depend on the domain, but will often include queries about the physical layout of points within the simulation space.

Example

A specialization of the **unstructured topology** for a two-dimensional mesh might define it in terms of vertices, edges, and elements.

Execution Levels



- **Main:** Top level of the application, running simultaneously on each **process**.
- **Control Model:** High-level structure of application execution.
- **Task:** The control model can launch many tasks. **Colors** identify which partition is being used.
- **Kernel:** Operation applied in parallel within one task execution.

Flog & Runtime

FleCSI Flog: Overview

- Flog is FleCSI's built-in, configurable, parallel logging utility (namespace `flecsi::flog`).
- If disabled at configure time, Flog symbols remain but produce no output.
- Typical use: aggregate task/thread output without interleaving, then serialize/flush periodically.

FleCSI Flog: Severity Levels

```
static flecsi::flog::tag mytag("mytag");

void some_task() noexcept {
    {
        flog::guard guard(mytag);
        flog(info) << "Info in mytag\n";
    }
    flog(warn) << "Warn\n";
    flog_assert(debug_check < 5, "Check value too large: " << debug_check);
    if (some_condition_failed)
        flog_fatal("Critical error in some_task");
}
```

- **Fatal is special:** `flog_fatal(...)` always emits and aborts, and is not suppressed by tags or strip levels; it can also print backtraces.
- **Assertions:** `flog_assert(...)` uses `flog_fatal` or does nothing with `NDEBUG`.

Running FleCSI

```
#include <flecsi/execution.hh>
#include <flecsi/runtime.hh>

int simulation(flecsi::scheduler &) {
    flog(info) << "Hello World\n";
    return 0;
}

int main() {
    const flecsi::run::dependencies_guard dg; // initializes MPI and/or Kokkos
    flecsi::runtime run;
    flecsi::flog::add_output_stream("clog", std::clog, true);
    return run.control<flecsi::run::call>(simulation);
}
```

Declaring `run` initializes FleCSI (with a default configuration). `add_output_stream` describes the output location for `flog`.
FleCSI then calls `simulation` via a built-in *control model* `call`.

Building FleCSI and Hands-on #1

Building FleCSI

FleCSI can be built using different methods depending on user needs:

Build Options

- **CMake**: Direct build using installed dependencies.
- **Spack**: Preferred method using the multi-platform package manager for easier dependency handling.



Spack

Hands-on #1: Building FleCSI

In this first hands-on we will

- Access your AWS session that features a pre-configured environment.
- Explore available Spack options for FleCSI.
- Build FleCSI using a provided Spack installation script.

Hands-on #1: Code

As a base for these hands-on exercises, we will use a simple heat-equation solver.

- Each hands-on will extend the initial code as new concepts are introduced.
- The application will compile at every module hands-on.
- The application will not fully execute in Module 3 (by design).
- A complete reference solution is available for each step in the repository if you need to catch up.

Heat Equation solver

Problem Setup

- 2-D heat diffusion on a uniform Cartesian grid.
- Unknown: temperature field $u(x, y, t)$.
- Fixed (Dirichlet) temperatures on all boundaries.
- Initial condition: Gaussian hot spot centered in the domain.

Governing Equation (simple form)

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u, \quad \alpha = \text{thermal diffusivity (constant and homogeneous)}$$

Heat Equation solver

Numerical Method

- Spatial discretization: 5-point stencil.
- Time integration: **implicit Backward Euler**

$$(I - \alpha \Delta t \nabla_h^2) u^{n+1} = u^n.$$

- Linear system solved with **Red–Black Gauss–Seidel**.
- Convergence checked using
 - change $\|u^{k+1} - u^k\|_\infty$ and
 - residual norm $\|Au^k - b\|_\infty$.

Heat Equation solver

Verification Using Analytical Solution

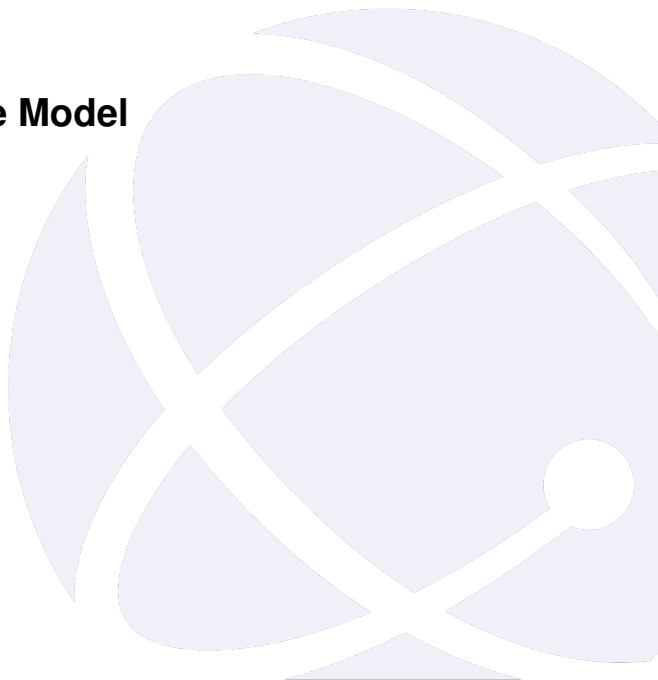
- Gaussian initial condition has an exact solution under diffusion.
- Code computes a relative L^2 error against this exact solution.
- Ensures correctness of discretization and solver.

Outputs

- CSV snapshots of $u(x, y, t)$.
- Can be visualized as 2D plots (python code included)

Module 2

Control Model and Runtime Model



Control Model Overview

- Organizes the application into smaller, modular entities:
 - **Control Points:** Logical stages in the application flow
 - **Actions:** Functions associated with control points, representing specific operations
- **Actions are not tasks.**
 - Actions run sequentially
 - Tasks can run in parallel and out of order
- Benefits of using the Control Model:
 - Simplifies the addition or removal of actions and task groups
 - Enhances code modularity and maintainability by allowing external dependency specifications

Control Model

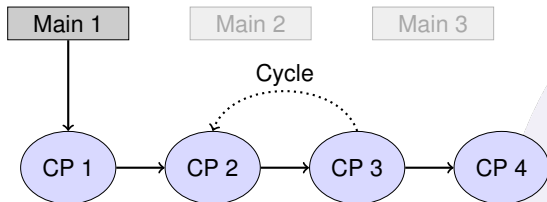
Main 1

Main 2

Main 3

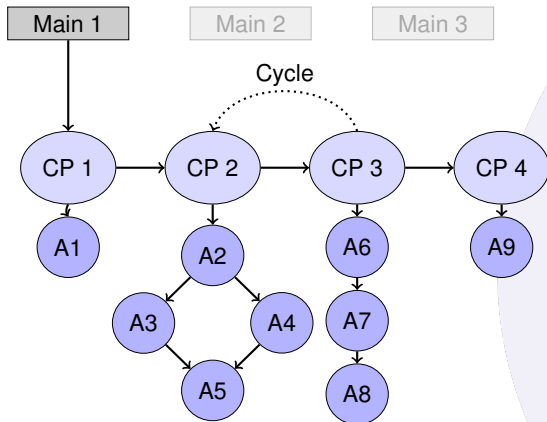
main and the control model attached to it is **replicated** on each process.

Control Model



Control points identify the **high-level stages of the application**. The execution order of the control points is statically defined. Cycles may be defined over any subset of the control points. They form a **control-flow graph (CFG)**.

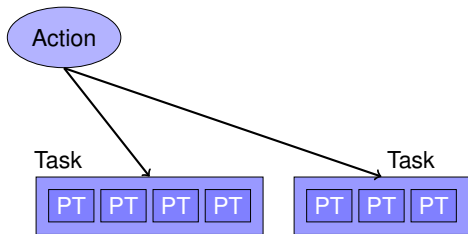
Control Model



Actions are **registered under control points**. Any two actions under a single control point may have a dependency defined between them. The actions under a single control point form a **directed acyclic graph (DAG)**. Actions allow composition of contributed packages. Actions are always executed in a **sequential** program order defined by:

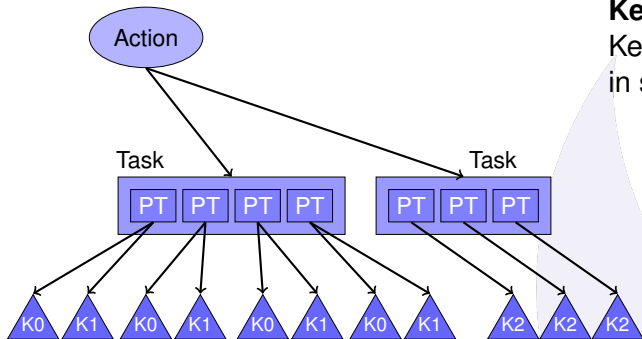
1. control point order
2. topological sorting the actions in the DAG under each control point

Execution Model



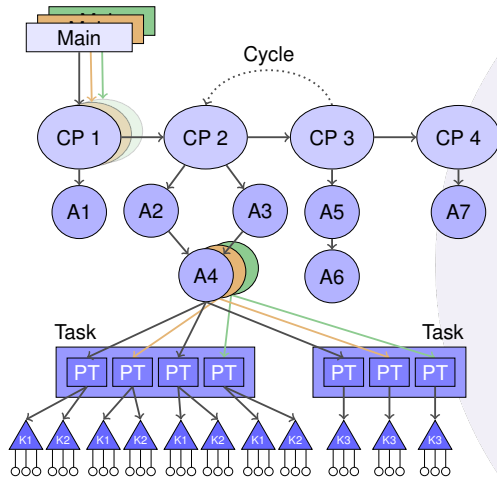
Tasks are launched by (the actions in) the control model. A task launch may be a **single** execution or an **index** launch of **Point Tasks**. Index tasks typically operate on data that are logically distributed over a partitioned address space. Dependency analysis allows tasks to be executed concurrently by the runtime.

Execution Model



Kernels are executed **inside a task**.
Kernels execute data-parallel operations
in some local address space.

Control Model and Execution Model



Runtime Model:

- main

Control Model:

- Control Points
- Actions

Driver:
- Once per process

Execution Model:

- Tasks, Point Tasks, Kernels
- Once per color

Hardware:

- CPU, OpenMP, GPU: thread

Control Model

Enumeration defining the control point identifiers.
This will be used to specialize the core control type.

```
enum class cp { initialize, advance, analyze, finalize };
```

Define labels for the control points. Using a function allows error checking.

```
inline const char* operator*(cp control_point) {  
    switch(control_point) {  
        case cp::initialize: return "initialize";  
        case cp::advance: return "advance";  
        case cp::analyze: return "analyze";  
        case cp::finalize: return "finalize";  
    }  
    flog_fatal("invalid control point");  
}
```

initialize

advance

analyze

finalize

Control Model

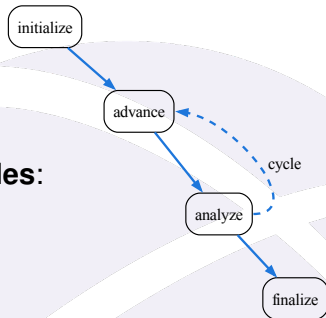
Inside the control policy we can define the behavior of **cycles**:

```
static bool cycle_control(control_policy& policy) {  
    return policy.step()++ < 5;  
}
```

We then select the enumeration of **control point**.

We create the list of control point based on the enumeration values.

```
using control_points_enum = cp;  
using control_points = list<point<cp::initialize>,  
                           cycle<cycle_control,  
                               point<cp::advance>,  
                               point<cp::analyze>>,  
                               point<cp::finalize>>>;
```



Control Model

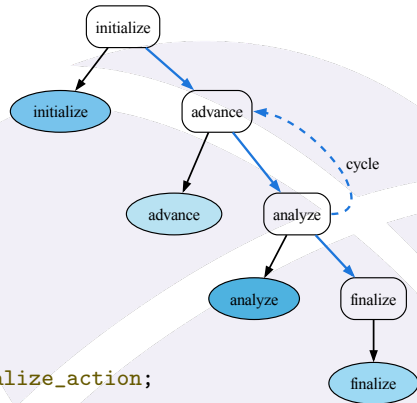
We then define the different control point functions.
Each can then be registered as an **action**.

For example:

```
void initialize(control_policy&) {  
    flog(info) << "initialize" << std::endl;  
}  
control::action<initialize, cp::initialize> initialize_action;
```

Which gives us the output:

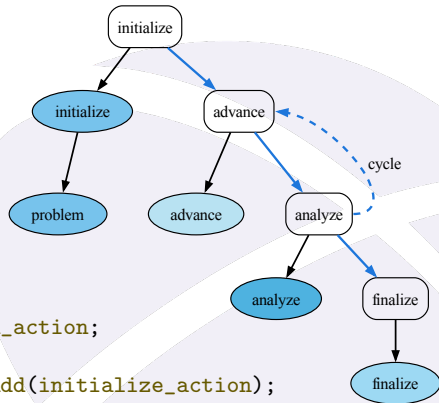
```
[info all p0] initialize  
[info all p0] advance  
[info all p0] analyze  
// ...  
[info all p0] advance  
[info all p0] analyze  
[info all p0] finalize
```



Control Model

We define **dependencies** between actions:

```
void problem(control_policy&) {}  
control::action<problem, cp::initialize> problem_action;  
  
inline const auto dep_problem = problem_action.add(initialize_action);
```



Control Model

All these parts are declared inside a `control_policy`:

```
struct control_policy : flecsi::run::control_base {  
    // ...  
};
```

Once we have created the control policy we can define a fully-qualified control type:

```
using control = flecsi::run::control<control_policy>;
```

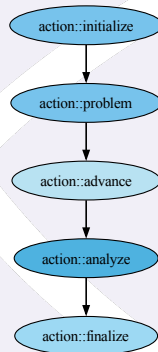
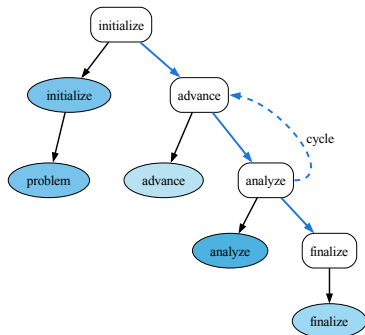
The actions have access to the **control policy**.

It is usually used to hold the field data needed for the application.

```
void my_action(control_policy &p) {  
    p.my_method();  
}
```

Control Model Visualization

The control model has support for **visualization**. It generates `graphviz` outputs that can be converted to images. It creates a view of the full graph or the selected ordering for the actions:



Runtime

Running FleCSI

Once the control model is implemented, we can run FleCSI on it:

```
int main() {  
    const flecsi::run::dependencies_guard dg;  
    flecsi::runtime run;  
    flecsi::flog::add_output_stream("clog", std::clog, true);  
    return run.control<control>();  
}
```

This will start our control model, running the actions in order.

Program Options

It is possible to create program options with FleCSI.

For example, the following adds support for `./prog --sx=15:`

```
flecsi::program_option<int> mx("Mesh X size",  
    "sizex,sx",  
    "Specify the size of the mesh in X dimension",  
    {{flecsi::option_default, 10}});
```

- `flecsi::getopt` parses options and produces help messages.
- `mx.value()` is the value of the program option if specified or defaulted.
- The user can check for value existence with `mx.has_value()`.
- `{flecsi::option_zero}` supports simple flags.

Hands-on #2

Hands-on #2: Control Model

In this second hands-on we will:

- Add a control model to a base application
- Define actions, cycles, and dependencies
- Update our runtime to use the new control model

Module 3

Data Model



Data Model: Fields and Topologies

- The **data model** defines how simulation data is
 - Declared and registered as **fields**
 - Associated with **topologies** (e.g., meshes, particles)
- **Fields** hold user data:
 - Declared by type and association
 - Registered once and accessed by tasks
- **Topologies** describe the structure of the domain:
 - Define entities (cells, vertices, edges, *etc.*)
 - Specify amounts and distribution of data
- Enables **automatic allocation, copying, and movement**

Concepts

Index Space

- List of entities (cells, nodes, *etc.*), each labeled with an *index point*.
- Conceptually, represents all entities in the problem, but
 - Data is **distributed** (as described by colors) and often duplicated (ghost elements).
 - No global ID for access.
- Practically, per-color sequences of owned and ghost entities.
- Each point task accesses one subdomain: locally, an index point is an array index.
- With Legion, colors need not correspond to processes (even in number, for increased parallelism); data can move as needed for load balancing

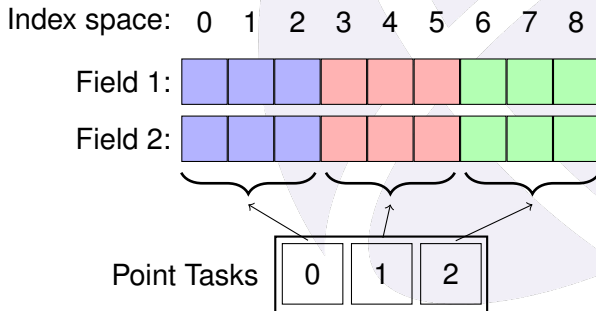
Index space:

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

Concepts

Fields

- Define a variable over an index space (*e.g.*, mass for a cell or momentum for a particle).
- Can represent scalars, vectors, or lists (*e.g.*, per material).
- Manage the memory associated with that variable.



Concepts

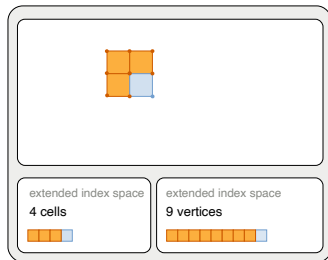
Ghost elements

- **Owned elements:** the subdomain proper
- **Ghost elements:** of another color, but copied locally (e.g., for stencil reads)
- **Shared elements:** owned and copied as ghosts elsewhere
- **Exclusive elements:** other owned elements
- Different access permissions will guide automatic **ghost copies**

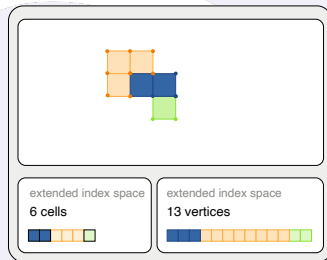
Concepts



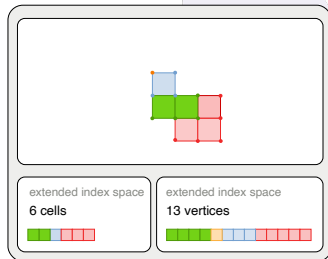
color 0 view



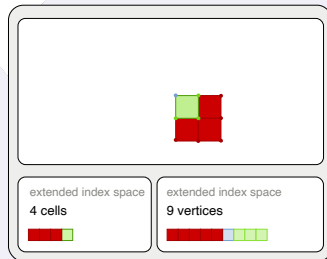
color 1 view



color 2 view



color 3 view



Fields

Fields

In the following example we are registering a field on a topology `topo_t`:

```
const flecsi::field<double>::definition<topo_t, topo_t::cells> mass_field;  
const flecsi::field<double>::definition<topo_t, topo_t::faces> massflux_field;
```

- We specify the type stored in the field, here `double`
- The `definition` specifies the **topology** and the **index space** (`topo_t::cells` or `topo_t::faces`) on which the field is registered.

Fields and Data Layout Trade-offs

```
struct cell_data_t { double mass; double energy; };  
const flecsi::field<cell_data_t>::definition<topo_t, topo_t::cells> cell_field;
```

Struct of Arrays (SoA)

- Enables independent access to each field (*e.g.*, `mass`)
- May incur extra overhead (*e.g.*, dependency analysis, indirections)

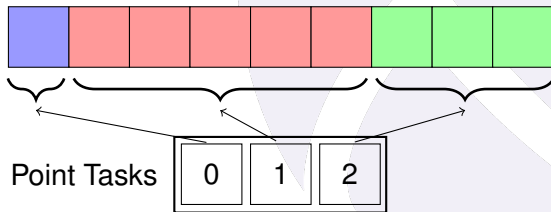
Array of Structs (AoS)

- Using one value (say, `mass`) requires loading all (also `energy`)
- May interfere with caching and vectorization

Data Layout: dense

```
flecsi::field<T/*, flecsi::data::dense*/>
```

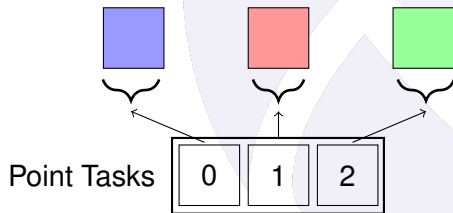
- Stores **one value** of type **T** per **index point**
- Most **common layout**; default if none specified
- **Use case**: uniform data per entity
- Example: Mass of each cell or momentum of each particle



Data Layout: single

```
flecsi::field<T, flecsi::data::single>
```

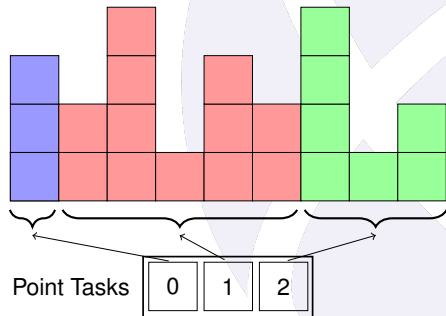
- Stores **one value** of type **T** per **color**
- Access does **not require an index point**
- **Use case:** flags or settings per subdomain
- Example: Enable a physics operator only for colors with relevant cells



Data Layout: ragged

```
flecsi::field<T, flecsi::data::ragged>
```

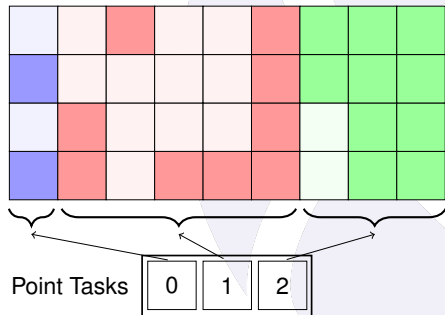
- Stores a **resizable array** of **T** per index point
- Analogous to `std::vector<T>` per index point
- **Use case:** variable-sized data (*e.g.*, material list per cell)
- Materials can **move** or change in count over time



Data Layout: sparse

```
flecsi::field<T, flecsi::data::sparse>
```

- Stores a **sparse set** of values per index point
- Conceptually like `std::map<std::size_t, T>`
- **More flexible** than `ragged`, allows missing elements
- **Use case**: materials with fixed IDs but not present in all cells



Data Layout: particle

```
flecsi::field<T, flecsi::data::particle>
```

- Stores an **unordered set** of **T** per color
- Index points merely serve to distinguish the objects present at any particular time
- **Use case:** tracking dynamic sets of particles
- Particles can change colors/subdomains

Allocation

Memory can be allocated only between tasks, so dynamic layouts must have enough **slack space** for one task. The amount can be specified by the application and can grow semi-automatically between tasks.

Topologies

Topology

- A **topology** represents the structure of a computational domain (*e.g.*, a mesh, grid, or particle distribution)
- It is composed of one or more **index spaces** that hold elements such as cells, vertices, or particles
- Some index spaces hold **physical** fields; others define **connectivity** or **relationships**
- Applications can use multiple topology instances (same or different types) at once
- Each topology belongs to a **category** (*e.g.*, **unstructured**, **global**) and a **specialization** (*e.g.*, burton, sph)
- Specializations **customize** things like mesh dimension and ghost support

Basic Topology Types

Topology categories are defined by FleCSI.
The most basic ones do not require specialization:

- **global**
 - Single index space, no colors/partitioning
 - Supports multiple reductions from a task
 - No support for **ragged** or **sparse** fields
- **index**
 - Single index space with one index point per color (as for the **single** layout)
 - Useful for metadata such as flags or integral quantities

Advanced Topology Types

- **user**

- Single index space sized per color
- No ghost support
- Useful for auxiliary per-color data containers

- **narray**

- Regular, multidimensional Cartesian layout
- Supports structured meshes with ghost and boundary regions
- Provides efficient access and connectivity

Advanced Topology Types

- **unstructured**

- Fully general graph-based topology
- Arbitrary adjacency relationships
- Arbitrary ghost copy patterns

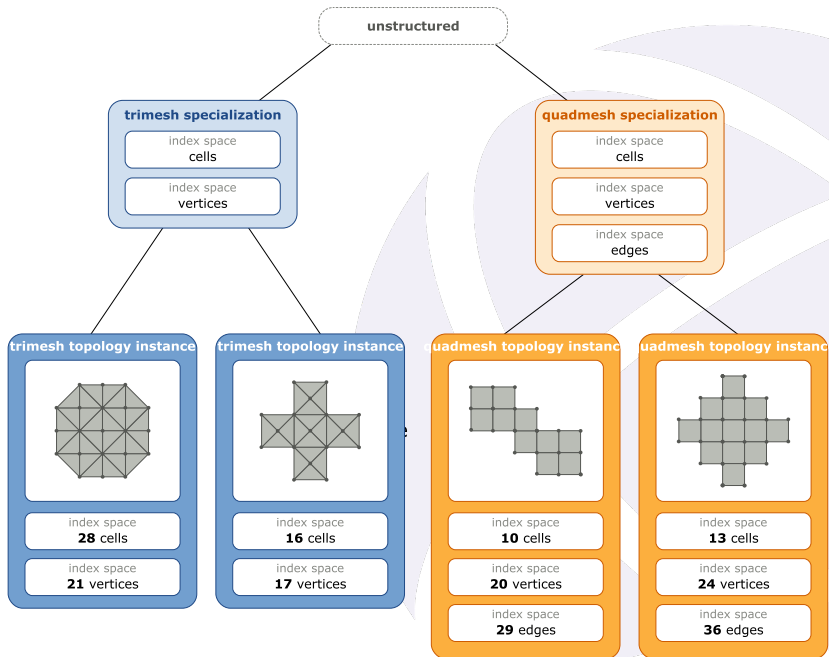
- **ntree**

- Hierarchical spatial decomposition (binary, quad-, or octree)
- Ideal for adaptive or particle-based domains
- Supports neighbor and range queries

topolog
categor

specialization

topology instances



Colorings and Partitioning

- A **coloring** defines indices and assigns colors (subdomains)
- Used to construct topology instances
- `global` and `index` use just an integer:

```
topo::global::topology pair(s, 2);    // 2 elements total  
topo::index::topology hydro(s, 42);  // 42 colors, 1 element each
```

- Complex topologies often use helper functions provided by the specialization:

```
canon::topology mesh(s, canon::mpi_coloring(s, "test.txt"));
```

- Topology instances must be created in the control model

Hands-on #3

Hands-on #3: Fields

In this third hands-on we will

- Incorporate a simple, pre-made specialization of the N-Array topology
- Add fields on this topology
- Not execute the **whole simulation** pending more tasks

Module 4

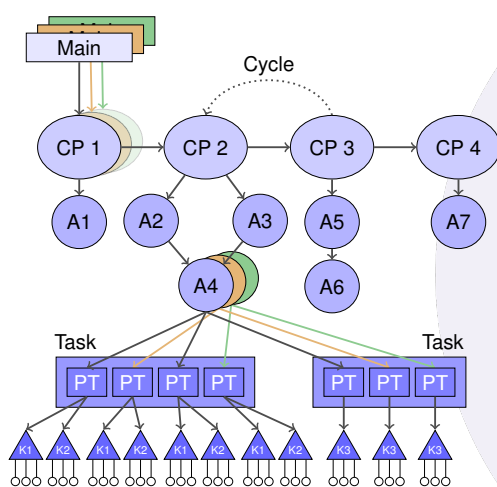
Execution Model



Execution Model — Bridging Parallelism Levels

- We have introduced
 - The **control model**: top-level orchestration
 - The **data model**: fields distributed across topologies
- Task launches bridge the two broad levels of the execution model:
 - **Driver** (main and control model)
 - **Tasks** (distributed parallelism)
- Tasks uniquely can
 - Declare data usage (privileges)
 - Access data and modify it

Execution Model — Bridging Parallelism Levels



Runtime Model:

- main

Control Model:

- Control Points
- Actions

Driver:
- Once per process

Execution Model:

- Tasks, Point Tasks, Kernels
- Once per color

Hardware:

- CPU, OpenMP, GPU: thread

Tasks

Tasks

- Tasks are **functions** that operate concurrently on a **distributed data structure**
- Task execution involves two sides:
 - **Action side**: provides arguments that **name** resources
 - **Task side**: defines parameters that provide **access** to resources (and cannot be constructed directly)
- The argument for a parameter is of a **different type** in general
- Clients can define composite task parameter and argument types

Field Access

- Fields are registered on **specializations**; every instance of the associated **topology type** has the field
- Conceptually similar to adding a member to a C++ `struct` ("`mesh.mass_field`"), but written `mass_field(mesh)`
- The field data can be accessed *only* by passing the resulting **field reference** to a task
- FleCSI automatically allocates memory for fields upon **first access**
- Fields are released when the associated **topology is destroyed**

Task: Action Side

```
int action(control_policy & cp) {  
    flecsi::scheduler & s = cp.scheduler();  
    s.execute<task1>(field1(*cp.ptr), field2(*cp.ptr));  
}
```

- Tasks are launched using the **scheduler**
 - available in control model actions
- **execute** takes the task function as a **template argument**
- Function arguments are **field references** and other inputs
- The task is executed **once per color**, asynchronously

Task: Signature

```
template<typename T>
void task2(const std::vector<T> &v,
          int i) noexcept
{ /* ... */ }
// ...
std::vector<int> v(...);
s.execute<task2<int>>(v, 10);
```

Templates & arguments

- Tasks can be templated, but cannot deduce template arguments
- Task arguments must be the same (value or field) on every process
- Since they are not invoked directly, tasks cannot throw exceptions and must be declared `noexcept`

Task Signature

```
using namespace flecsi;  
void task3(exec::cpu s,  
          field<int>::accessor<rw, rw, ro> acc_1,  
          field<double, data::ragged>::mutator<rw, na, na> mut_2) noexcept  
{ auto me = s.launch().index; /* ... */ }  
// ...  
s.execute<task3>(exec::on, /* ... */);
```

Execution Space

- Specifies where the task will execute and describes the launch
- Several options are possible: `cpu` (default), `gpu`, `omp`, or `accelerator`
- Later we will define one task for **multiple** execution spaces
- Use fixed token `exec::on` on the action side

Task Signature

```
using namespace flecsi;  
void task3(exec::cpu s,  
           field<int>::accessor<rw, rw, ro> acc_1,  
           field<double, data::ragged>::mutator<rw, na, na> mut_2) noexcept  
{ /* ... */ }
```

Accessors

- A task accepts **field references** as arguments via **accessors**
- When the task is launched, the (allocated) memory is provided to the task through the accessors
- We will soon show how to use them in the task
- There are also **topology accessors** for structural information with a specialization-defined interface

Task Signature

```
using namespace flecsi;  
void task3(exec::cpu s,  
           field<int>::accessor<rw, rw, ro> acc_1,  
           field<double, data::ragged>::mutator<rw, na, na> mut_2) noexcept  
{ /* ... */ }
```

Mutators

- **Accessors** can read/write values but **cannot add or remove** them
- Layouts that support dynamic modification use **mutators**
- Mutators exist for Ragged, Sparse, and Particle layouts

Task Signature

```
using namespace flecsi;  
void task3(exec::cpu s,  
           field<int>::accessor<rw, rw, ro> acc_1,  
           field<double, data::ragged>::mutator<rw, na, na> mut_2) noexcept  
{ /* ... */ }
```

Privileges

- **na**: no access, **ro**: read-only, **wo**: write-only, and **rw**: read-write
- Accessors and mutators declare the task's **privileges** on each field
- Privileges are used by the runtime to **enforce task order**
- First access must use a **write-only** accessor/mutator to initialize the field

Privileges

- **Ghost copies** are controlled by **multiple privileges** on appropriate index spaces:
<exclusive> *<shared, ghost>* *<exclusive, shared, ghost>*
- Pointers/references become **const**-qualified if **no write privileges**
- **Undefined behavior** if code writes to regions lacking write permission

Ghost copy

A ghost copy occurs between any two tasks that operate on the same field where

- the first has write access to the shared elements,
- the second has read access to the ghost elements, and
- neither the first nor any intervening task has write access to the ghost elements.

Futures

```
int source() noexcept {return 10;}  
int filter(flecsi::future<int> f) noexcept {return f.get() + 1;}  
// ...  
flecsi::future<int> f = s.execute<source>();  
s.execute<filter>(f);
```

- Launching a task returns a **future**
- The task's return value becomes available later

Warning

- Calling `f.get()` outside a task forces **serialization** of execution
- Better practice: **pass futures to other tasks** instead
- Alternative: call `get` only after launching several more tasks

Reduction

```
auto err = s.reduce<squared_error, exec::fold::sum>(...);
```

- The `reduce` call performs a reduction over multiple point task results.
- It returns a **future** for the result of the reduction operation.
- Reduction operations are defined in the `exec::fold` namespace:
 - `sum`, `product`, `min`, and `max`
- You can define a reduction type with
 - `combine`: a function to merge two values.
 - `identity`: the neutral element for the operation.

Further Task Semantic

- **MPI tasks** are a special type of task that execute MPI code independently of the backend.
- When an MPI task runs, **Legion execution is globally synchronized.**
- This allows safe use of native MPI calls inside FleCSI.
- MPI tasks **break Legion's asynchronous flow** and reduce parallel efficiency.
- Usage is **discouraged in normal execution**, but may be acceptable for **initialization or setup phases.**

Inside the Task

Accessor

```
using namespace flecsi;  
void task4(field<int>::accessor<rw> acc) noexcept {  
    acc[0] *= 2;  
    for(auto& v: acc.span())  
        v += 10;  
}
```

Dense Accessors

- Can be accessed as an array with `operator[]` or `operator()`
- Provide a `span()` which is an iterable view

Layouts

```
using namespace flecsi;  
void task5(field<int, data::ragged>::accessor<rw> acc) noexcept {  
    for(auto &x : acc[0]) ++x;  
    for(auto r : acc) ++r[0];  
}
```

Ragged Accessors

- **Rows** per index point can be accessed with `operator[]` or iteration
- Each is a `span<int>` (with the same two operations), or is `std::map`-like for `data::sparse`

Mutator

```
using namespace flecsi;  
void task6(field<int, data::ragged>::mutator<rw> mut) noexcept {  
    int i = 0;  
    for(auto r : mut) {  
        r.resize(i, i);  
        ++i;  
    }  
}
```

Mutators

- `ragged` now emulates `std::vector<T>` at each point
- `sparse` provides `std::map<std::size_t, T>::try_emplace`, *etc.*
- `particle` emulates `std::hive` (C++26)

Hands-on #4

Hands-on #4: Tasks

In this fourth hands-on we will

- Define our own tasks and execute them
- Use accessors and mutators as task parameters
- Use reductions and futures

Module 5

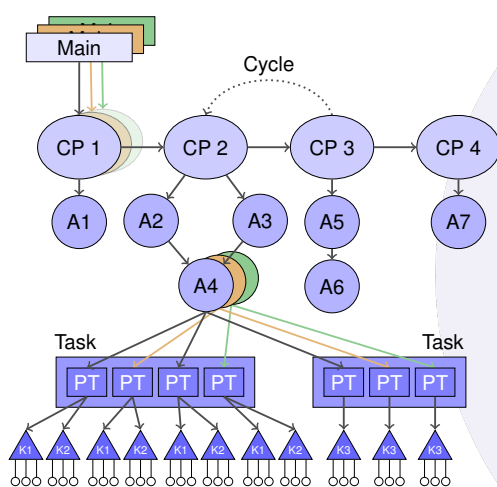
On-Node Parallelism



On-Node Parallelism - Getting Performance

- We have introduced
 - The **control model**: top-level orchestration
 - The **data model**: fields and topologies
 - The **execution model**: tasks to distribute and organize work
- **On-node parallelism** supports leveraging different accelerators
- It is available through **kernels**
- **Within each task**, FleCSI provides interfaces to use **Kokkos** for
 - Fine-grain, **thread-level parallelism**
 - Efficient use of **hardware backends** (e.g., OpenMP, CUDA)

Execution Model — Bridging Parallelism Levels



Runtime Model:

- main

Control Model:

- Control Points
- Actions

Driver:
- Once per process

Execution Model:

- Tasks, Point Tasks, Kernels
- Once per color

Hardware:

- CPU, OpenMP, GPU: thread

Executor and Loops

On-Node Parallelism

- FleCSI tasks always run on the host but can launch **kernels** to exploit **fine-grained, on-node parallelism**
- **Kokkos** maps them to **hardware threads**
- Field data is **placed in the correct memory space**

Executor

The **executor API** provides a consistent interface on execution spaces:

- `forall`, like `Kokkos::parallel_for`
- `reduceall`, like `Kokkos::parallel_reduce`

```
void modify(exec::accelerator s, /* ... */ ) noexcept {  
    s.executor(). /* ... */  
}
```

On-Node Parallelism

FleCSI supports the selection of different execution spaces in the task signature:

- `exec::cpu`: execute on host CPU
- `exec::omp`: execute on several host CPUs with OpenMP
- `exec::gpu`: execute with GPU support
- `exec::accelerator`: depends on the configuration of Kokkos.
 - The architecture will be, if available, `gpu > omp > cpu`.
 - If `exec::gpu` or `exec::omp` is used explicitly but not supported by the Kokkos build, the code will not compile.

Simple Parallel Loop

```
void modify(exec::accelerator s, field<double>::accessor<rw> p) noexcept {  
    s.executor().forall(x, p.span()) {  
        x += 1.0;  
    };  
}
```

- The **macro** `forall` executes a **parallel loop** over a range
- `x` is the implicit loop variable
- `p.span()` defines the iteration space
- The loop body is a **lambda expression** followed by a semicolon

Reduction Kernels

```
void reduce(exec::accelerator s, field<float>::accessor<ro> p) noexcept {  
    auto res = s.executor().reduceall(  
        x, up, p.span(), exec::fold::max, float) {  
        up(x);  
    };  
}
```

- `reduceall` performs a **parallel reduction**
- `up` is an **accumulator** called as a function on each `float`
- Same reduction operations as for task return values

Ranges

FleCSI provides **GPU-compatible** utilities to iterate on fields:

```
void task(exec::cpu, field<float>::accessor<rw> p) noexcept {  
    for(auto v: p.span())  
        v += 1.0  
}
```

span

- Can be retrieved directly from accessor/mutator
- Non-owning view similar to `std::span`
- Provide access to 1D array with any container providing size and pointer

`std::ranges::iota_view` can be used to make a range of integers

Advanced Ranges

```
void task(exec::cpu, field<float>::accessor<rw> p) noexcept {  
    int is = 3, js = 4;  
    flecsi::util::mdspan<float, 2> mds(p.span().data(), {js, is});  
    for(int i = 0; i < is - 1; ++i)  
        for(int j = 0; j < js; ++j)  
            mds[i][j] = mds[i+1][j];  
}
```

mdspan and mdcolex

- Based on a subset of `std::mdspan` from C++23
- Multi-dimensional indexing
- `mdcolex` is similar to `mdspan` with reversed indices

Advanced Ranges

`mdiota_view` produces `mdspan` indices for `forall` or `reduce` loops:

```
flecsi::util::mdspan<float, 2> mds(p.span().data(), {js, is});
s.executor().forall(
    mi,
    mdiota_view(
        mds,
        exec::full_range(),
        exec::prefix_range{is-1}
    )) {
    auto [j, i] = mi;
    mds[i][j] = mds[i+1][j];
};
```

`full_range`, `prefix_range`, and `sub_range` select a subrectangle

Executor Features

```
s.executor()  
  .threads<64, 1>()  
  .named("named forall")  
  .for_each(p.span(),  
    [p] FLECSI_INLINE_TARGET (auto c) {  
      p[c] += 2;  
    });
```

- `threads<...>()`: controls number of threads used by Kokkos
- `named("...")`: labels a kernel (useful for debugging and profiling)
- `for_each`: function-object version of `forall`
 - Compared to `forall`, `for_each` allows to specify the lambda and its capture/parameter
 - Similarly `reduce` for `reduceall`
- Device functions need to be annotated with `FLECSI_INLINE_TARGET`

GPU Pitfalls

```
void task(exec::gpu s, field<float>::accessor<rw> p) noexcept {  
    p[0] = 0; // Error: cannot be accessed on the host  
    std::vector<float> v{1.0,2.0,3.0,4.0};  
    s.executor().forall(x, p.span()) {  
        x += v[0]; // Error: host std::vector cannot be accessed  
    };  
}
```

Caution!

- The field data is put on the GPU for you and is not accessible on the host
- Data declared outside the `forall` are copied, if possible (trivially copyable)
- You can use the `for_each` construct to control the capture list

Certain **topology data** can be used outside kernels (to make ranges)

Portable Tasks

Portable Tasks

- A **portable task** is composed of several **variants** targeting different **execution spaces**
- Each variant is a (C++) specialization of a **static member function template**
- FleCSI automatically selects a variant when the task is launched
- Variants that are not implemented can be explicitly deleted to prevent their use

Task Template Structure

```
struct task_variants {  
    template<class S>  
    static void task(S s, /* ... */ ) noexcept {  
        s.executor().forall(/* ... */) { /* ... */};  
    }  
};  
// ...  
s.execute<task_variants>(exec::on, /* ... */);
```

- The name `task` for the function is predefined
- To give a generic interface, `S` cannot be used in other parameters
- For technical reasons, portable tasks **cannot** be implemented as non-member or overloaded functions
- The same `forall` can work for all variants

Example: Portable Task Variants

```
struct task_variants {  
    template<class S>  
        static void task(S, /* ... */ ) noexcept = delete;  
};  
  
template<>  
void task_variants::task(exec::cpu s, /* ... */ ) noexcept { /* ... */ }  
  
template<>  
void task_variants::task(exec::omp s, /* ... */ ) noexcept { /* ... */ }
```

- The primary `task` template is deleted to remove the default implementation
- Specialized variants are provided for `exec::cpu` and `exec::omp`
- Tasks cannot be launched on other architectures (e.g., `exec::gpu`)

Hands-on #5

Hands-on #5: on-node parallelism

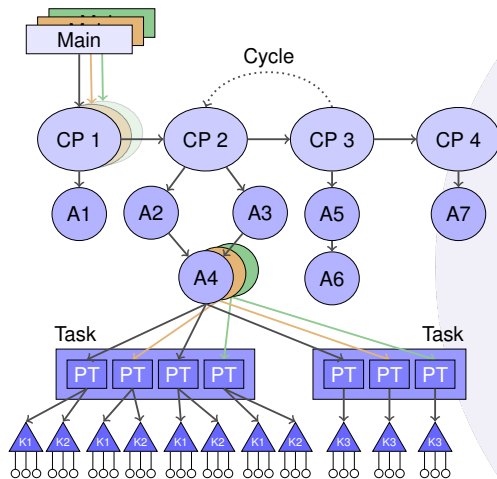
In this last hands-on we will:

- Use the FleCSI parallel loops (`forall` and `reduceall`)
- Use advanced ranges
- Use a portable task

Conclusion



FleCSI Model



Runtime Model:

- main

Control Model:

- Control Points
- Actions

Driver:

- Once per process

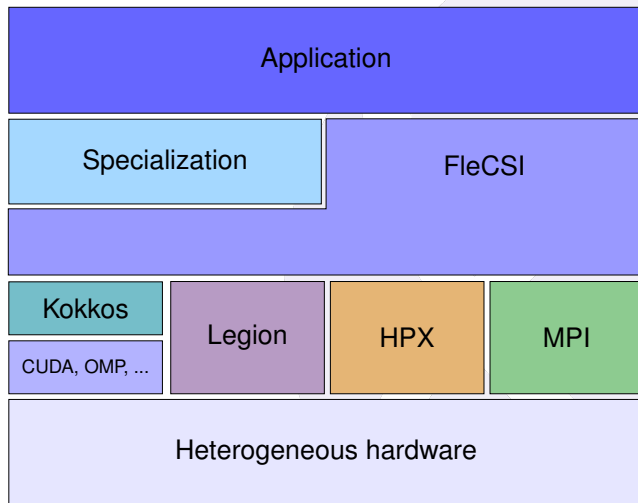
Execution Model:

- Tasks, Point Tasks, Kernels
- Once per color

Hardware:

- CPU, OpenMP, GPU: thread

FleCSI Model



Resources

Available Resources:

- <https://github.com/flecsi/flecsi>: FleCSI GitHub repository
 - Create **issue** or use the **GitHub discussion board**
- <https://www.flecsi.org/>:
 - FleCSI Tutorial + User guide
 - User API Reference

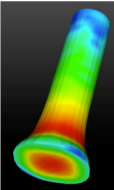


Version: 2.4.1

- CONTENTS:
- Overview
- Build & Install
- Tutorial
- User Guide
- API Reference
- Developer Guide
- Contributors

🏠 / Introduction [View page source](#)

Introduction



FleCSI is a compile-time configurable framework designed to support multi-physics application development. As such, FleCSI provides a very general set of infrastructure design patterns that can be specialized and extended to suit the needs of a broad variety of solver and data requirements. FleCSI currently supports multi-dimensional mesh topology, geometry, and adjacency information, as well as n-dimensional hashed-tree data structures, graph partitioning interfaces, and dependency closures.

FleCSI introduces a functional programming model with control, execution, and data abstractions that are consistent both with MPI and with state-of-the-art, task-based runtimes such as Legion and Charm++. The abstraction layer insulates developers from the underlying runtime, while allowing support for multiple runtime systems including conventional models like asynchronous MPI. The overall design is described further in

Fig. 1 Taylor Anvil Impact

FleCSI contributors

Benjamin Bergen, Nick Moss, Irina Demeshko, **Davis Herring**, Marc Charest, **Julien Loiseau**, Navamita Ray, Jonathan Graham, Hartmut Kaiser, Li-Ta Lo, Karen Tsai, Charles Ferenbaugh, **Richard Berger**, John Wohlbier, Jonas Lippuner, Wei Wu, Andrew Reisner, Christoph Junghans, **Scott Pakin**, Brendan K. Krueger, Lukas Spies, Sumathi Lakshmiranganatha,

Max Ortner, Pascal Grosset, David Gunter, **Maxim Moraru**, Galen Shipman, Jiajia Waters, Scot Halverson, Onur Çaylak, Peter Brady, Philipp V. F. Edelmann, Mason Delan, Brandon Keim, Christopher Malone, Alex Villa, Daniel Holladay, Dani Barrack, Nikunj Gupta, Ondřej Čertík, Robert Bird, Melissa Rasmussen